

연결 리스트 실습 과제 설명

이 문서는 연결 리스트의 삽입, 삭제, 연결, 역순 변환, 탐색 기능을 포함한 실습 코드의 구조와 각 함수의 역할을 설명합니다.

📋 실습 목표

- 연결 리스트 기본 연산 구현
- 동적 메모리 할당을 이용한 노드 생성 및 관리
- 리스트 간 연결 및 역순 처리
- 특정 값 탐색 기능 구현

📁 파일 구성

- `main()` 함수: 실습 시나리오 실행
- `create_node()`: 노드 생성
- `insert_node()`: 노드 삽입
- `remove_node()`: 노드 삭제
- `display()`: 리스트 출력
- `concat()`: 두 리스트 연결
- `reverse()`: 리스트 역순 변환
- `search()`: 리스트에서 값 탐색

🔑 주요 함수 설명

`Node* create_node(int data, Node* link)`

- 새 노드를 생성하고 포인터를 반환합니다.
- 매개변수: 데이터 값, 다음 노드 포인터

`void insert_node(Node** head, Node* pre, Node* new_node)`

- 리스트 맨 앞 또는 특정 위치 뒤에 노드 삽입
- `pre == NULL`일 경우, 맨 앞에 삽입

`void remove_node(Node** head, Node* pre, Node* removed)`

- 리스트에서 노드를 삭제하고 메모리 해제
- `pre == NULL`이면 맨 앞 노드를 삭제

`void display(Node* head)`

- 리스트 노드를 순서대로 출력 (예: `10 -> 20 -> NULL`)

`Node* concat(Node* head1, Node* head2)`

- 두 리스트를 이어붙여 새로운 리스트 반환

Node* reverse(Node* head)

- 리스트를 뒤집어 반환

Node* search(Node* head, int x)

- 특정 값을 가진 노드를 반환 (없으면 NULL 반환)

실행 결과 예시

```
30 -> 20 -> 10 -> NULL
20 -> 10 -> NULL
80 -> 70 -> 60 -> NULL
20 -> 10 -> 80 -> 70 -> 60 -> NULL
60 -> 70 -> 80 -> 10 -> 20 -> NULL
탐색성공: 20
```